CNCF Project Focus

Episode #5

# OPEN TELEMETRY

Vendor-Neutral Unified Observability

Christian Dussol

**“The application is slow !”**

**Where do you look ?**

---

**Metrics** → CPU & Memory are fine

**Logs** → "No errors found"

**Traces** → "Can't find the request"

**3 tools. 3 SDKs. 3 dashboards.**

❌ **Zero correlation**
❌ **Locked into each tool's instrumentation**

# 3 Pillars of Observability

**Metrics**

"What is happening?"
*Numbers. Rates. Percentiles*

**Logs**

"What happened here?"
*Events. Errors. Context*

**Traces**

"What was the journey?"
*Full path. Every service. Every hop*

**Each alone = partial visibility**

**All 3 correlated = full observability for distributed systems**

# Metrics

```
http_request_duration_seconds {
  service="payment-svc", status="500"
} = 2.3
```

✅ Detects problems (alerting)
✅ Trends over time
✅ Cheap to store

❌ Which request failed?
❌ Why was it slow?

**Smoke alarm**
detects fire, not which room

# Logs

```
{
  "level": "ERROR",
  "service": "payment-svc",
  "message": "Gateway timeout",
  "order_id": "ORD-4821",
  "duration_ms": 5023
}
```

✅ Error details + stack traces
✅ Business context (order ID, user ID)

❌ Cause or symptom?
❌ What called this service?

**Crime scene report**
one event, no timeline

# Traces

```
order-svc ──────────────────────────── 350ms
 ├── validate ──────────────────────── 5ms
 ├── payment-svc ────────────────────── 200ms ⚠️
 │    └── call_gateway ──────────── 180ms 🔴
 └── inventory-svc ──────────────── 120ms
```

✅ Full path across all services
✅ Latency per operation
✅ Exact bottleneck location

❌ One-time or pattern? (need metrics)
❌ Business context? (need logs)

**GPS tracker**
full journey, every delay

# The game changer: Correlation

**One Trace ID connects everything**

**Trace ID:** abc-123-def

📈 METRIC → "p99 spike on payment-svc"

↕

📋 LOG → "Gateway timeout: ORD-4821"

↕

🔍 TRACE → "call_gateway = 180ms 🔴"

**Before:** Alert → manual search → hours
**After:** Alert → click on Trace ID → minutes

# What is OpenTelemetry

**Born in 2019**
Merger of OpenTracing + OpenCensus

**Status**
CNCF Incubating (Graduation under review)

Among the most active CNCF projects

Thousands of contributors from 1,000+ orgs

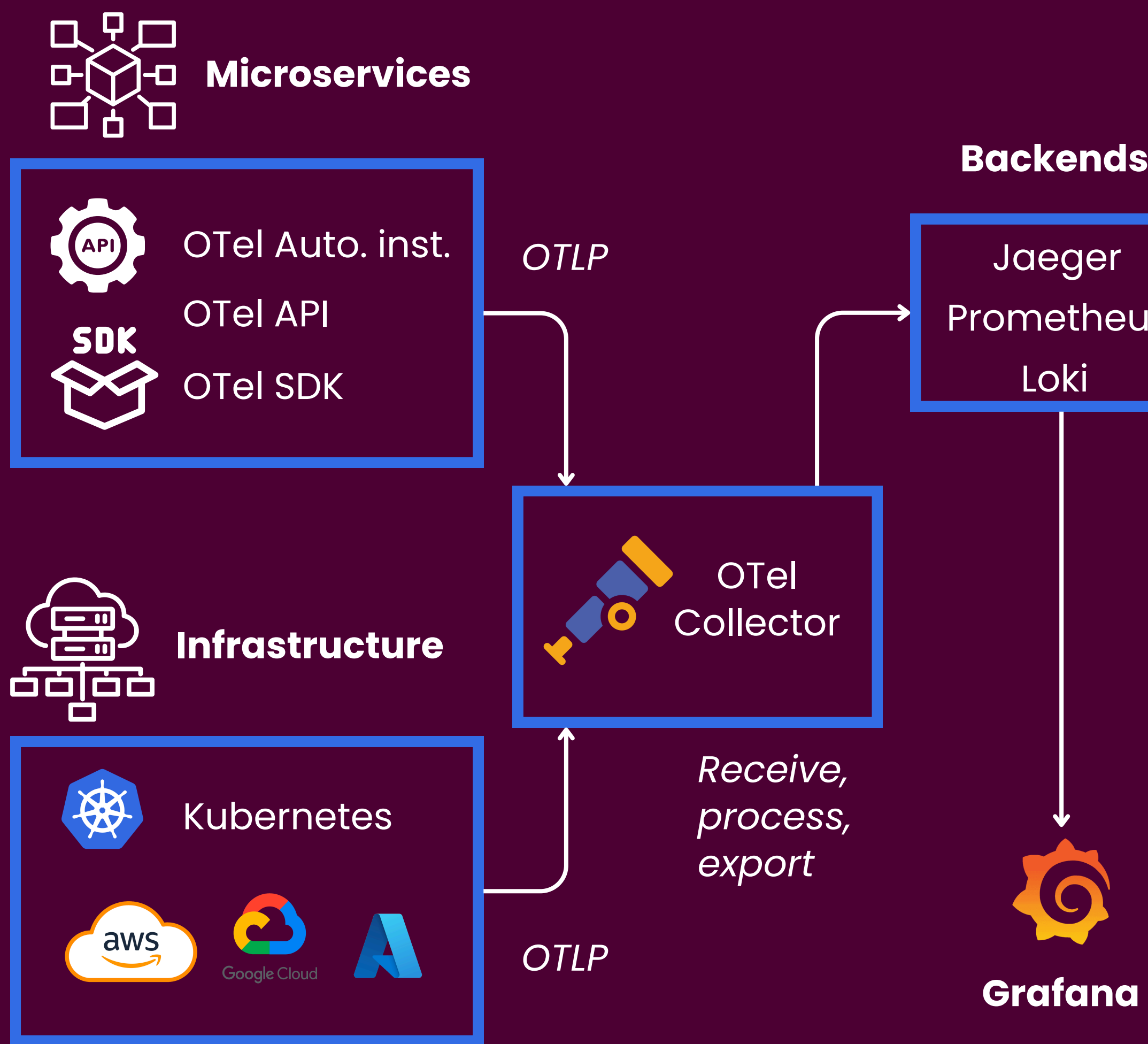
SDKs in 11+ languages

In production at

GitHub ebay shopify

**Not a tool. An open standard**

# Architecture




**4 core components**

- API (Vendor Neutral interfaces)
- SDK (11+ languages)
- OTLP (OpenTelemetry Protocol, one wire format for all signals)
- Collector (Unifies correlated signals into one pipeline)

# Vendor neutral

## Before OTel

App → Vendor SDK → Locked
Switch = rewrite everything

## After OTel

App → OTel SDK → OTel Collector → ANY backend
Switch = change config file

One pipeline. All signals linked. Any backend.

---

K8s decoupled workloads from infrastructure.

**OTel decouples instrumentation from backends**

Instrument once. Export anywhere.
**Your data. Your choice.**

# Know your tool

**OpenTelemetry is built for**

✅ Microservices / distributed systems
✅ Multi-cloud / hybrid
✅ Vendor independence
✅ Signal correlation (one Trace ID)

**OpenTelemetry is NOT designed for**

❌ Simple monolith (overkill)
❌ Replacing backends (OTel collects, not stores)
❌ Zero-effort setup (needs instrumentation strategy)

# OTel + PROMETHEUS

| | Prometheus | OTel |
|---|---|---|
| **Signals** | Metrics only | Metrics + Logs + Traces |
| **Model** | Pull (scrape) | Push (OTLP) |
| **Strength** | PromQL + Alerting | Vendor neutrality + correlation |

**Not competing. Composing**

OTel as the ingestion standard
Prometheus as one of the backends

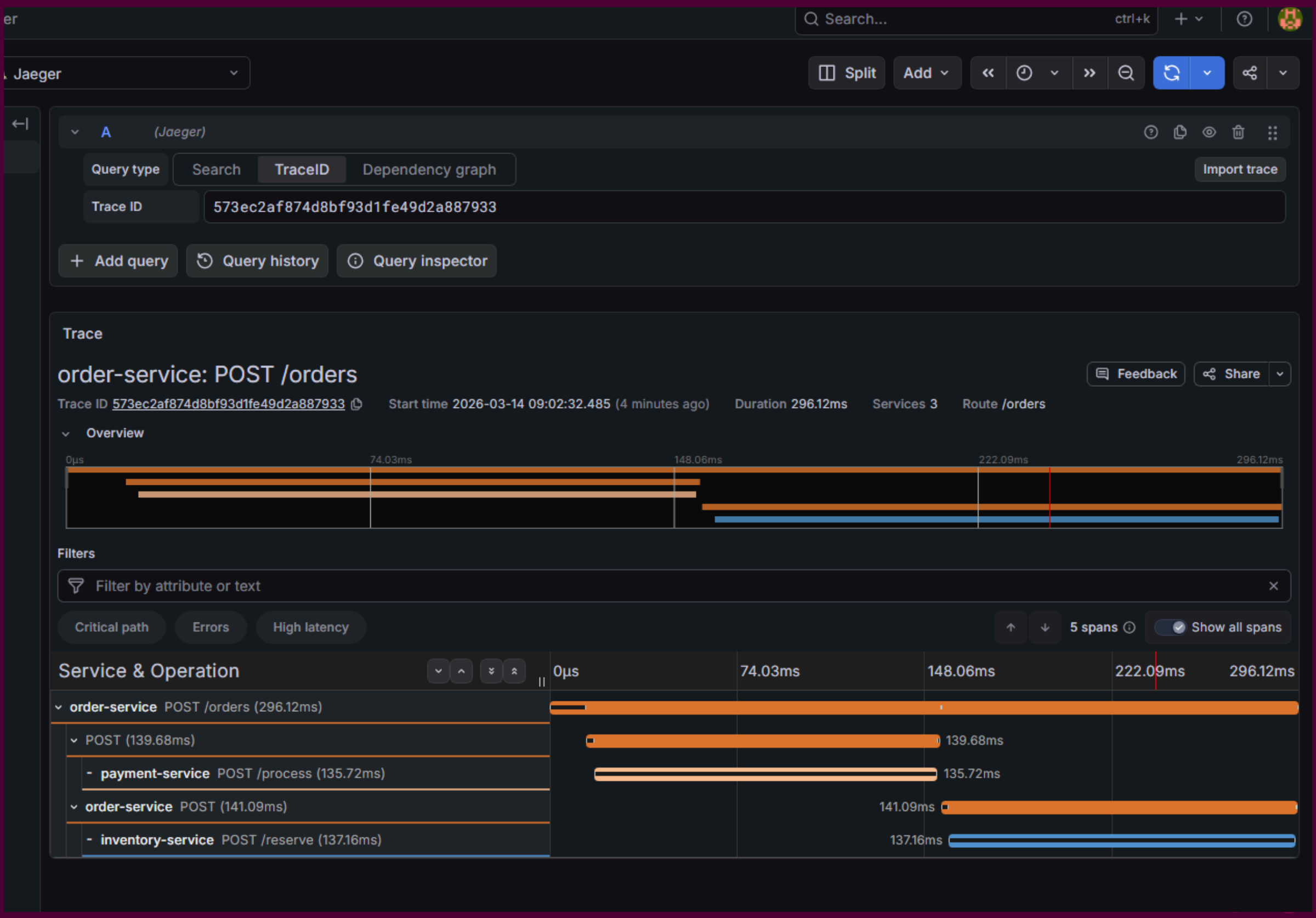
# Educational GitHub Repository

## Explore my learning toolkit

bitly

github.com/christian-dussol-cloud-native/opentelemetry

Complete hands-on tutorial, including:

- ✅ OTel Collector on Kubernetes (Helm)
- ✅ Auto-instrumentation (zero-code)
- ✅ 3-service distributed tracing demo
- ✅ Head vs. tail sampling (FinOps)
- ✅ Kyverno policies for cost governance

# Educational GitHub Repository